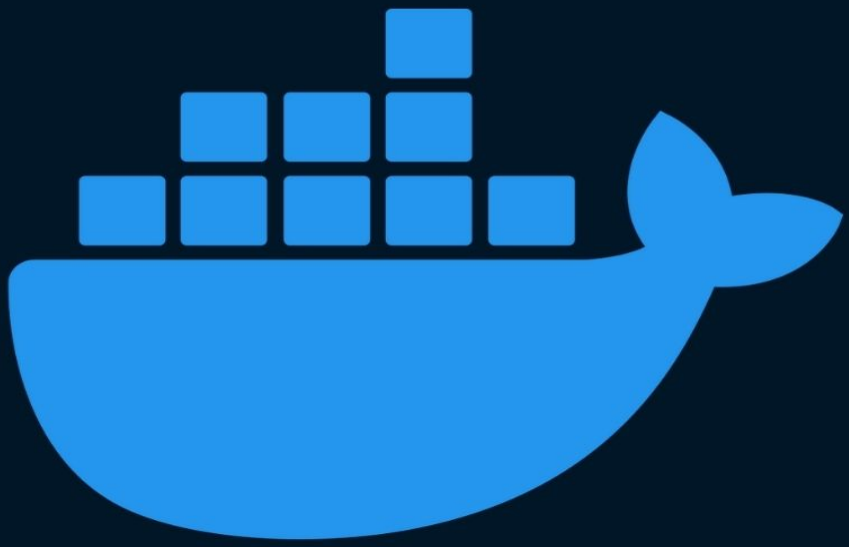
AN OPENSOURCE EBOOK

INTRODUCTION TO



docker®

Bobby Iliev

Table of Contents

About the book

- **This version was published on August 19 2024**

This is an introduction to Docker ebook that will help you learn the basics of Docker and how to start using containers for your SysOps, DevOps, and Dev projects. No matter if you are a DevOps/SysOps engineer, developer, or just a Linux enthusiast, you will most likely have to use Docker at some point in your career.

The guide is suitable for anyone working as a developer, system administrator, or a DevOps engineer and wants to learn the basics of Docker.

About the author

My name is Bobby Iliev, and I have been working as a Linux DevOps Engineer since 2014. I am an avid Linux lover and supporter of the open-source movement philosophy. I am always doing that which I cannot do in order that I may learn how to do it, and I believe in sharing knowledge.

I think it's essential always to keep professional and surround yourself with good people, work hard, and be nice to everyone. You have to perform at a consistently higher level than others. That's the mark of a true professional.

For more information, please visit my blog at https://bobbyiliev.com, follow me on Twitter @bobbyiliev_ and YouTube.

Sponsors

This book is made possible thanks to these fantastic companies!

Materialize

The Streaming Database for Real-time Analytics.

Materialize is a reactive database that delivers incremental view updates. Materialize helps developers easily build with streaming data using standard SQL.

DigitalOcean

DigitalOcean is a cloud services platform delivering the simplicity developers love and businesses trust to run production applications at scale.

It provides highly available, secure, and scalable compute, storage, and networking solutions that help developers build great software faster.

Founded in 2012 with offices in New York and Cambridge, MA, DigitalOcean offers transparent and affordable pricing, an elegant user interface, and one of the largest libraries of open source resources available.

For more information, please visit https://www.digitalocean.com or follow @digitalocean on Twitter.

If you are new to DigitalOcean, you can get a free $100 credit and spin up your own servers via this referral link here:

Free $200 Credit For DigitalOcean

DevDojo

The DevDojo is a resource to learn all things web development and web design. Learn on your lunch break or wake up and enjoy a cup of coffee with us to learn something new.

Join this developer community, and we can all learn together, build together, and grow together.

Join DevDojo

For more information, please visit https://www.devdojo.com or follow @thedevdojo on Twitter.

Ebook PDF Generation Tool

This ebook was generated by Ibis developed by Mohamed Said.

Ibis is a PHP tool that helps you write eBooks in markdown.

Book Cover

The cover for this ebook was created with Canva.com.

If you ever need to create a graphic, poster, invitation, logo, presentation – or anything that looks good — give Canva a go.

License

MIT License

Copyright (c) 2020 Bobby Iliev

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Chapter 1: Introduction to Docker

What is Docker?

Docker is an open-source platform that automates the deployment, scaling, and management of applications using containerization technology. It allows developers to package applications and their dependencies into standardized units called containers, which can run consistently across different environments.

Key Concepts:

1. **Containerization**: A lightweight form of virtualization that packages applications and their dependencies together.
2. **Docker Engine**: The runtime that allows you to build and run containers.
3. **Docker Image**: A read-only template used to create containers.
4. **Docker Container**: A runnable instance of a Docker image.
5. **Docker Hub**: A cloud-based registry for storing and sharing Docker images.

Why Use Docker?

Docker offers numerous advantages for developers and operations teams:

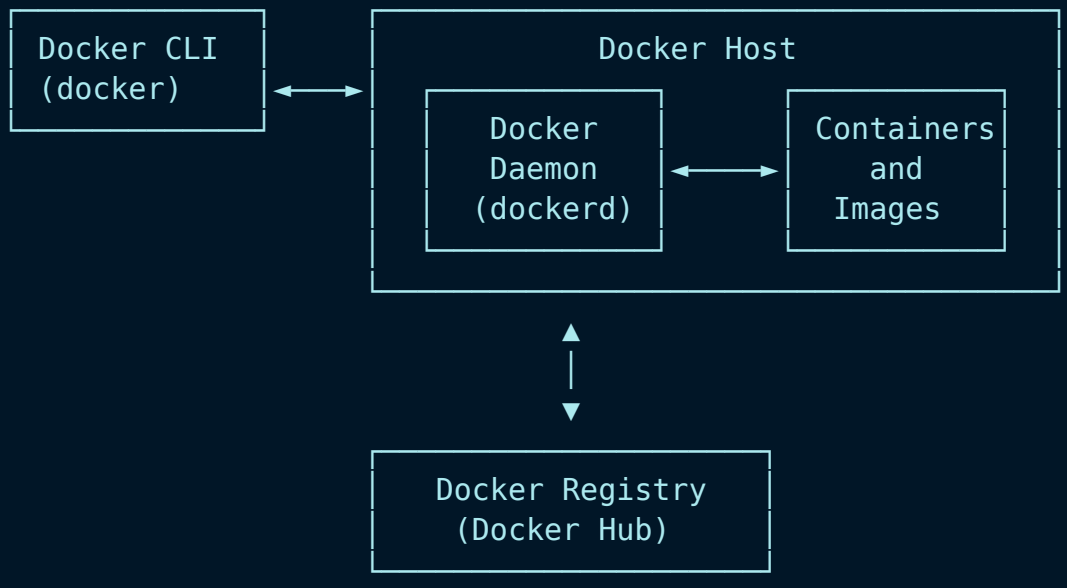
1. **Consistency**: Ensures applications run the same way in development, testing, and production environments.
2. **Isolation**: Containers are isolated from each other and the host system, improving security and reducing conflicts.
3. **Portability**: Containers can run on any system that supports Docker, regardless of the underlying infrastructure.
4. **Efficiency**: Containers share the host system's OS kernel, making them more lightweight than traditional virtual machines.
5. **Scalability**: Easy to scale applications horizontally by running multiple containers.
6. **Version Control**: Docker images can be versioned, allowing for easy rollbacks and updates.

Docker Architecture

Docker uses a client-server architecture:

1. **Docker Client**: The primary way users interact with Docker through the command line interface (CLI).
2. **Docker Host**: The machine running the Docker daemon (dockerd).
3. **Docker Daemon**: Manages Docker objects like images, containers, networks, and volumes.
4. **Docker Registry**: Stores Docker images (e.g., Docker Hub).

Here's a simplified diagram of the Docker architecture:



Containers vs. Virtual Machines

While both containers and virtual machines (VMs) are used for isolating applications, they differ in several key aspects:

Aspect	Containers	Virtual Machines
OS	Share host OS kernel	Run full OS and kernel
Resource Usage	Lightweight, minimal overhead	Higher resource usage
Boot Time	Seconds	Minutes
Isolation	Process-level isolation	Full isolation
Portability	Highly portable across different OSes	Less portable, OS-dependent
Performance	Near-native performance	Slight performance overhead
Storage	Typically smaller (MBs)	Larger (GBs)

Basic Docker Workflow

1. **Build**: Create a Dockerfile that defines your application and its dependencies.
2. **Ship**: Push your Docker image to a registry like Docker Hub.
3. **Run**: Pull the image and run it as a container on any Docker-enabled host.

Here's a simple example of this workflow:

```
# Build an image
docker build -t myapp:v1 .

# Ship the image to Docker Hub
docker push username/myapp:v1

# Run the container
docker run -d -p 8080:80 username/myapp:v1
```

Docker Components

1. **Dockerfile**: A text file containing instructions to build a Docker image.
2. **Docker Compose**: A tool for defining and running multi-container Docker applications.
3. **Docker Swarm**: Docker's native clustering and orchestration solution.
4. **Docker Network**: Facilitates communication between Docker containers.
5. **Docker Volume**: Provides persistent storage for container data.

Use Cases for Docker

1. **Microservices Architecture**: Deploy and scale individual services independently.
2. **Continuous Integration/Continuous Deployment (CI/CD)**: Streamline development and deployment processes.
3. **Development Environments**: Create consistent development environments across teams.
4. **Application Isolation**: Run multiple versions of an application on the same host.
5. **Legacy Application Migration**: Containerize legacy applications for easier management and deployment.

Conclusion

Docker has revolutionized how applications are developed, shipped, and run. By providing a standardized way to package and deploy applications, Docker addresses many of the challenges faced in modern software development and operations. As we progress through this book, we'll dive deeper into each aspect of Docker, providing you with the knowledge and skills to leverage this powerful technology effectively.

Chapter 2: Installing Docker

Installing Docker is the first step in your journey with containerization. This chapter will guide you through the process of installing Docker on various operating systems, troubleshooting common issues, and verifying your installation.

Docker Editions

Before we begin, it's important to understand the different Docker editions available:

1. **Docker Engine - Community**: Free, open-source Docker platform suitable for developers and small teams.
2. **Docker Engine - Enterprise**: Designed for enterprise development and IT teams building, running, and operating business-critical applications at scale.
3. **Docker Desktop**: An easy-to-install application for Mac or Windows environments that includes Docker Engine, Docker CLI client, Docker Compose, Docker Content Trust, Kubernetes, and Credential Helper.

For most users, Docker Engine - Community or Docker Desktop will be sufficient.

Installing Docker on Linux

Docker runs natively on Linux, making it the ideal platform for Docker containers. There are two main methods to install Docker on Linux: using the convenience script or manual installation for specific distributions.

Method 1: Using the Docker Installation Script (Recommended for Quick Setup)

Docker provides a convenient script that automatically detects your Linux distribution and installs Docker for you. This method is quick and works across many Linux distributions:

1. Run the following command to download and execute the Docker installation script:

```
wget -qO- https://get.docker.com | sh
```

2. Once the installation is complete, start the Docker service:

```
sudo systemctl start docker
```

3. Enable Docker to start on boot:

```
sudo systemctl enable docker
```

This method is ideal for quick setups and testing environments. However, for production environments, you might want to consider the manual installation method for more control over the process.

Method 2: Manual Installation for Specific Distributions

For more control over the installation process or if you prefer to follow distribution-specific steps, you can manually install Docker. Here are instructions for popular Linux distributions:

Docker runs natively on Linux, making it the ideal platform for Docker containers. Here's how to install Docker on popular Linux distributions:

Ubuntu

1. Update your package index:

```
sudo apt-get update
```

2. Install prerequisites:

```
sudo apt-get install apt-transport-https ca-certificates
curl software-properties-common
```

3. Add Docker's official GPG key:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg |
sudo apt-key add -
```

4. Set up the stable repository:

```
sudo add-apt-repository "deb [arch=amd64]
https://download.docker.com/linux/ubuntu $(lsb_release -
cs) stable"
```

5. Update the package index again:

```
sudo apt-get update
```

6. Install Docker:

```
sudo apt-get install docker-ce docker-ce-cli containerd.io
```

CentOS

1. Install required packages:

```
sudo yum install -y yum-utils device-mapper-persistent-
data lvm2
```

2. Add Docker repository:

```
sudo yum-config-manager --add-repo
https://download.docker.com/linux/centos/docker-ce.repo
```

3. Install Docker:

```
sudo yum install docker-ce docker-ce-cli containerd.io
```

4. Start and enable Docker:

```
sudo systemctl start docker
sudo systemctl enable docker
```

Other Linux Distributions

For other Linux distributions, refer to the official Docker documentation: https://docs.docker.com/engine/install/

Installing Docker on macOS

For macOS, the easiest way to install Docker is by using Docker Desktop:

1. Download Docker Desktop for Mac from the official Docker website: https://www.docker.com/products/docker-desktop

2. Double-click the downloaded `.dmg` file and drag the Docker icon to your Applications folder.

3. Open Docker from your Applications folder.

4. Follow the on-screen instructions to complete the installation.

Installing Docker on Windows

For Windows 10 Pro, Enterprise, or Education editions, you can install Docker Desktop:

1. Download Docker Desktop for Windows from the official Docker website: https://www.docker.com/products/docker-desktop

2. Double-click the installer to run it.

3. Follow the installation wizard to complete the installation.

4. Once installed, Docker Desktop will start automatically.

For Windows 10 Home or older versions of Windows, you can use Docker Toolbox, which uses Oracle VirtualBox to run Docker:

1. Download Docker Toolbox from: https://github.com/docker/toolbox/releases

2. Run the installer and follow the installation wizard.

3. Once installed, use the Docker Quickstart Terminal to interact with Docker.

Post-Installation Steps

After installing Docker, there are a few steps you should take:

1. Verify the installation:

   ```
   docker version
   docker run hello-world
   ```

2. Configure Docker to start on boot (Linux only):

   ```
   sudo systemctl enable docker
   ```

3. Add your user to the docker group to run Docker commands without sudo (Linux only):

   ```
   sudo usermod -aG docker $USER
   ```

   Note: You'll need to log out and back in for this change to take effect.

Docker Desktop vs Docker Engine

It's important to understand the difference between Docker Desktop and Docker Engine:

- **Docker Desktop** is a user-friendly application that includes Docker Engine, Docker CLI client, Docker Compose, Docker Content Trust, Kubernetes, and Credential Helper. It's designed for easy installation and use on Mac and Windows.

- **Docker Engine** is the core Docker runtime available for Linux systems. It doesn't come with the additional tools included in Docker Desktop but can be installed alongside them separately.

Troubleshooting Common Installation Issues

1. **Permission denied**: If you encounter "permission denied" errors, ensure you've added your user to the docker group or are using sudo.

2. **Docker daemon not running**: On Linux, try starting the Docker service: `sudo systemctl start docker`

3. **Conflict with VirtualBox (Windows)**: Ensure Hyper-V is enabled for Docker Desktop, or use Docker Toolbox if you need to keep using VirtualBox.

4. **Insufficient system resources**: Docker Desktop requires at least 4GB of RAM. Increase your system's or virtual machine's allocated RAM if needed.

Updating Docker

To update Docker:

- On Linux, use your package manager (e.g., `apt-get upgrade docker-ce` on Ubuntu)
- On Mac and Windows, Docker Desktop will notify you of updates automatically

This is a sample from "Introduction to Docker" by Bobby Iliev.

For more information, Click here.